

KAN-22

Block "3" - Endpoint búsqueda mentores (Backend)

- Implementar servicio backend para consultar mentores según criterios de búsqueda enviados desde frontend.
- Incluir lógica de filtrado, ordenamiento y respuesta estructurada de resultados.

- - - - -

El objetivo principal de este bloque consiste en implementar la lógica del lado del servidor (Backend) para gestionar el flujo de consulta, filtrado y ordenamiento de los perfiles de mentores disponibles en la plataforma. El sistema debe procesar dinámicamente las peticiones enviadas por el Frontend y cumplir estrictamente con las siguientes reglas de negocio:

1. **Filtro de seguridad por Rol y Estado:** El endpoint no debe exponer información abierta de la base de datos; únicamente debe listar aquellos perfiles que pertenezcan a usuarios con `rol = 2` (correspondiente a Mentores) y cuyo `estado` sea estrictamente `'activo'`.
2. **Filtrado Dinámico y Parcial:** El sistema debe evaluar de forma opcional los criterios de búsqueda enviados desde la URL, permitiendo búsquedas exactas para el campo `ciclo` y búsquedas parciales (`LIKE`) para campos de texto como `carrera` y `habilidades`.
3. **Ordenamiento Dinámico Sanitizado:** Se debe permitir al Frontend definir bajo qué columna estructurar el orden de los resultados, implementando mecanismos de control para evitar inyecciones SQL en las cláusulas de ordenamiento.

El desarrollo de este requerimiento se concentró exclusivamente en dos archivos del proyecto, garantizando un impacto aislado y limpio en la arquitectura existente:

`app/Models/Perfil.php` (Adaptación del Modelo):

Se implementó formalmente la relación inversa `usuario()` utilizando el método `BelongsTo` apuntando hacia el modelo de usuarios (`Usuario::class` o `User::class`). Esta adición corrigió un comportamiento de error crítico (Error 500) al mapear la clave foránea `usuario_id`, permitiendo que Eloquent realice la carga ambiciosa (*eager loading*) de los datos de identidad de cada mentor de forma transparente y eficiente.

`app/Http/Controllers/PerfilController.php` (Lógica de Filtrado y API):

Dentro de este controlador se concentró toda la lógica de negocio del KAN-22 mediante el método estructurado:

- **`index(Request $request)`:** 1. Inicializa la consulta mediante un método `whereHas('usuario')` para encapsular las reglas de negocio de rol y estado activo, acoplando simultáneamente los datos mediante `with('usuario')`. 2. Evalúa de manera condicional la existencia de parámetros en la petición (`$request->has()`) y concatena limpiamente los operadores de condición (`where` y `where like`) para la carrera, el ciclo y las habilidades. 3. Ejecuta una lógica de ordenamiento dinámico que reasigna el orden por defecto al campo `fecha_actualizacion` (previniendo fallos por columnas inexistentes como `created_at`) y sanitiza las entradas del Frontend validando los campos y direcciones contra arrays de valores permitidos (`in_array`). 4. Segmenta la respuesta final utilizando el método `paginate(5)`, inyectando la estructura de control de páginas requerida para la navegación.

Aunque las rutas del sistema ya estaban predefinidas en el repositorio por el equipo, la validación del flujo se completó exitosamente gracias a la lógica implementada en PerfilController.php. El endpoint de consulta opera bajo el enfoque de peticiones HTTP estandarizadas:

- **GET /api/perfiles** (Ubicado en routes/api.php y apuntando a PerfilController@index): Recibe los parámetros de búsqueda en la Query String de la URL, procesa los filtros de manera acumulativa en el servidor y retorna una respuesta JSON estructurada con la data de los mentores emparejada con su respectiva información de usuario.

La verificación y el aseguramiento de la calidad del código se realizaron mediante dos fases de pruebas en caliente:

1. **Simulación de Peticiones API (Postman / Thunder Client):** Se validó el endpoint **GET /api/perfiles** enviando diferentes combinaciones de parámetros de búsqueda (**?carrera=Software, ?ciclo=5, ?habilidad=Laravel**). El sistema respondió de forma exitosa arrojando un estado **200 OK**, demostrando que la paginación calcula dinámicamente los metadatos de enlaces (**next_page_url, prev_page_url**) y totales en base a las coincidencias reales.
2. **Persistencia Física de Datos (HeidiSQL):** Se inspeccionó la base de datos local bajo el entorno Laragon insertando registros de prueba controlados para verificar los filtros. Se constató el correcto funcionamiento del filtro de rol al comprobar que los usuarios con rol de estudiante son omitidos del listado, y se validó que el ordenamiento responde fielmente a las actualizaciones reflejadas en la columna **fecha_actualizacion**.